



وثيقة تحليل وتصميم قاعدة البيانات

الجامعة: جامعة الحكمة

القسم: قسم تقنية معلومات

المقرر: قواعد بيانات (Database)

أعضاء المجموعة:

.الطالب: إبراهيم جبر علي العولقي

.الطالب: رفيق عادل الوليدي

.الطالب: خطاب منصور البريهي

.الطالب: خالد وليد العمري

.الطالب: أنس إبراهيم الصيفي

جدول المحتويات

أ. الهدف ونطاق النظام3	
ب. الكيانات وخصائصها4	
ج. قواعد العمل9	
هـ. القيود وقواعد العمل	
د. مخطط المفاهيم14	 (Conceptual ER Diagram)
و. مخطط العلاقة	 Relational Schema Diagram

أ. الهدف من النظام

يهدف نظام إدارة الجامعة إلى إنشاء قاعدة بيانات متكاملة لإدارة البيانات الأكاديمية والإدارية والخدمات المقدمة للطلاب وأعضاء هيئة التدريس، وتنظيم المعلومات المتعلقة بالكليات والأقسام والتخصصات والمقررات والفصول الدراسية والمحاضرات والاختبارات والحضور والغياب والخدمات الطلابية.

ب. الكيانات وخصائصها

2.1 University • الجامعة –

الوصف: يمثل الجامعة التي تحتوي على الكليات.

الخصائص: – University_ID المفتاح الأساسي؛ University_Name؛ University_Phone؛ University_Email

2.2 Colleges • الكليات –

الوصف: تمثل الكليات التابعة للجامعة.

الخصائص: – College_ID المفتاح الأساسي؛ College_Name؛ – University_ID مفتاح أجنبي

2.3 Departments • الأقسام –

الوصف: تمثل الأقسام الموجودة داخل الكليات.

الخصائص: – Department_ID المفتاح الأساسي؛ Department_Name؛ – College_ID مفتاح أجنبي

2.4 Major • التخصص –

الوصف: يمثل التخصصات الأكاديمية التي يقدمها القسم.

الخصائص: – Major_ID المفتاح الأساسي؛ Major_Name؛ – Department_ID مفتاح أجنبي

2.5 Students • الطلاب –

الوصف: يمثل الطلاب المسجلين في الجامعة.

الخصائص: – Student_ID المفتاح الأساسي؛ First_Name؛ Last_Name؛ Date_of_Birth؛ Phone؛ Email؛ – Major_ID مفتاح أجنبي

2.6 Faculty Members • أعضاء هيئة التدريس –

الوصف: يمثل أعضاء هيئة التدريس العاملين في الأقسام.

الخصائص: – Faculty_ID المفتاح الأساسي؛ First_Name؛ Last_Name؛ Academic_Rank؛ Email؛ Phone؛ – Department_ID مفتاح أجنبي

2.7 Courses • المقررات –

الوصف: تمثل المقررات الدراسية التي تقدمها الجامعة.

الخصائص: Course_ID – المفتاح الأساسي؛ Course_Name؛ Credit_Hours؛ Department_ID – مفتاح أجنبي

– 2.8 Rooms • القاعات

الوصف: تمثل القاعات المستخدمة للمحاضرات والاختبارات.

الخصائص: Room_ID – المفتاح الأساسي؛ Room_Number؛ Capacity؛ Building_ID – مفتاح أجنبي

– 2.9 Academic Semesters • الفصول الدراسية

الوصف: تمثل الفصول الدراسية التي يتم خلالها تقديم المقررات.

الخصائص: Semester_ID – المفتاح الأساسي؛ Semester_Name؛ Start_Date؛ End_Date

– 2.10 Buildings • المباني

الوصف: تمثل مباني الجامعة التي تحتوي على القاعات.

الخصائص: Building_ID – المفتاح الأساسي؛ Building_Name؛ Location

– 2.11 Lectures • المحاضرات

الوصف: تمثل المحاضرات الخاصة بالمقررات.

الخصائص: Lecture_ID – المفتاح الأساسي؛ Lecture_Date؛ Start_Time؛ End_Time؛ Course_ID – مفتاح أجنبي؛

Room_ID – مفتاح أجنبي؛ Semester_ID – مفتاح أجنبي

– 2.12 Exams • الاختبارات

الوصف: تمثل الاختبارات الخاصة بالمقررات.

الخصائص: Exam_ID – المفتاح الأساسي؛ Exam_Date؛ Exam_Time؛ Course_ID – مفتاح أجنبي؛ Room_ID – مفتاح أجنبي؛

Semester_ID – مفتاح أجنبي

– 2.13 Attendance & Absence • الحضور والغياب

الوصف: يمثل سجل حضور وغياب الطلاب في المحاضرات، ويُعامل كـ Weak Entity.

الخصائص: Student_ID – مفتاح أجنبي؛ Lecture_ID – مفتاح أجنبي؛ Attendance_Date؛ Attendance_Status؛

Notes؛ Check_In_Time؛ المركب: (Student_ID, Lecture_ID)

– 2.14 Department Heads • رؤساء الأقسام

الوصف: يمثل رؤساء الأقسام المسؤولين عن إدارة الأقسام.

الخصائص: Head_ID – المفتاح الأساسي؛ Head_Name؛ Department_ID – مفتاح أجنبي

– 2.15 Scholarships • المنح

الوصف: تمثل المنح الدراسية التي يحصل عليها الطلاب.

الخصائص: Scholarship_ID – المفتاح الأساسي؛ Scholarship_Name؛ Amount؛ Start_Date؛ End_Date؛ Student_ID

– مفتاح أجنبي

– 2.16 Finance • المالية

الوصف: تمثل المعاملات المالية المتعلقة بالطلاب.

الخصائص: Finance_ID – المفتاح الأساسي؛ Student_ID – مفتاح أجنبي؛ Amount؛ Transaction_Date؛

Transaction_Type

– 2.17 Library • المكتبة

الوصف: تمثل خدمات ومعلومات المكتبة الجامعية.

الخصائص: Library_ID – المفتاح الأساسي؛ Library_Name؛ Location

– 2.18 Student Affairs • شؤون الطلاب

الوصف: تمثل الخدمات والطلبات المقدمة للطلاب من شؤون الطلاب.

الخصائص: Status: Request_Date: Request_Type؛ Student_ID مفتاح أجنبي؛ Affairs_ID المفتاح الأساسي؛ -

2.19 Graduation Projects • مشاريع التخرج

الوصف: تمثل مشاريع التخرج التي يعمل عليها الطلاب.

الخصائص: Project_ID المفتاح الأساسي؛ Project_Name؛ Project_Description؛ Project_Date

2.20 Academic Registration • التسجيل الأكاديمي

الوصف: يمثل تسجيل الطالب في المقررات خلال الفصل الدراسي.

الخصائص: Registration_ID المفتاح الأساسي؛ Student_ID مفتاح أجنبي؛ Course_ID مفتاح أجنبي؛ Semester_ID

Registration_Date؛ مفتاح أجنبي؛

ج. العلاقات وكيفية تنفيذها

University • مع Colleges: علاقة 1:N؛ الجامعة قد تحتوي عدة كليات، وكل كلية تتبع جامعة واحدة. تُنقذ بإضافة University_ID
Colleges. كمفتاح أجنبي في.

Colleges • مع Departments: علاقة 1:N؛ الكلية تحتوي عدة أقسام، وكل قسم يتبع كلية واحدة. تُنقذ بإضافة College_ID في
Departments.

Departments • مع Major: علاقة 1:N؛ القسم يقدم عدة تخصصات، وكل تخصص يتبع قسمًا واحدًا. تُنقذ بإضافة
Department_ID في Major.

Major • مع Students: علاقة 1:N؛ التخصص يضم عدة طلاب، والطالب يتبع تخصصًا واحدًا. تُنقذ بإضافة Major_ID في
Students.

Departments • مع Faculty_Members: علاقة 1:N؛ القسم يعمل فيه عدة أعضاء هيئة تدريس، وكل عضو يتبع قسمًا. تُنقذ
بإضافة Department_ID في Faculty_Members.

Faculty_Members • مع Courses: علاقة M:N؛ العضو يدرّس عدة مقررات والمقرر يدرّسه عدة أعضاء. تُنقذ بجدول وسيط
Faculty_Course_Semester يحتوي Faculty_ID و Course_ID و Semester_ID.

Courses • مع Academic_Semesters: علاقة M:N؛ المقرر يطرح في عدة فصول والفصل يحتوي عدة مقررات. تُنقذ بجدول
Faculty_Course_Semester أو جدول ربط طرح المقرر.

Students • مع Courses: علاقة M:N؛ الطالب يسجل عدة مقررات والمقرر يسجل فيه عدة طلاب. تُنقذ بجدول
Academic_Registration مع Student_ID و Course_ID و Semester_ID.

Courses • مع Lectures: علاقة 1:N؛ المقرر يحتوي عدة محاضرات وكل محاضرة تتبع مقررًا. يُحفظ Course_ID في Lectures.

Buildings • مع Rooms: علاقة 1:N؛ المبنى يحتوي عدة قاعات وكل قاعة تقع في مبنى. يُحفظ Building_ID في Rooms.

Lectures • مع Rooms و Academic_Semesters: كل محاضرة تُعقد في قاعة واحدة وفصل دراسي واحد. تُحفظ Room_ID و
Semester_ID في Lectures.

Courses • مع Exams: علاقة 1:N؛ المقرر يحتوي عدة اختبارات وكل اختبار يتبع مقررًا. يُحفظ Course_ID في Exams، مع
Semester_ID و Room_ID.

Students • مع Lectures: علاقة M:N؛ الطالب يحضر عدة محاضرات والمحاضرة يحضرها عدة طلاب. تُنقذ بالكيان الضعيف
Attendance_Absence ومفتاحه المركب (Student_ID, Lecture_ID).

Departments • مع Department_Heads: علاقة 1:1؛ لكل قسم رئيس وفق قواعد الجامعة، ويُحفظ Department_ID في
Department_Heads.

Students • مع Scholarships و Finance و Student_Affairs: علاقات 1:N؛ الطالب قد يملك عدة منح ومعاملات وطلبات،
وتحتوي الكيانات التابعة Student_ID.

Students • مع Graduation_Projects و Faculty_Members: Graduation_Projects: علاقات M:N؛ تُنقذ بجدول ربط
Student_Project و Faculty_Project_Supervision.

د. قواعد العمل والتكامل والقيود

• 4. الجامعة يمكن أن تحتوي على عدة كليات، وكل كلية تتبع جامعة واحدة.

- 5. الكلية يمكن أن تحتوي على عدة أقسام، وكل قسم يتبع كلية واحدة.
- 6. القسم يمكن أن يقدم عدة تخصصات، وكل تخصص يتبع قسمًا واحدًا.
- 7. التخصص يمكن أن يضم عدة طلاب، والطلاب يتبع تخصصًا واحدًا.
- 8. أعضاء هيئة التدريس يعملون في الأقسام.
- 9. عضو هيئة التدريس يمكن أن يدرّس عدة مقررات، والمقرر يمكن أن يدرّسه أكثر من عضو هيئة تدريس.
- 10. المقرر يمكن أن يُطرح في عدة فصول دراسية، والفصل الدراسي يمكن أن يحتوي على عدة مقررات.
- 11. الطالب يمكن أن يسجل عدة مقررات خلال الفصل الدراسي.
- 12. المقرر يمكن أن يحتوي على عدة محاضرات، وكل محاضرة تتبع مقررًا واحدًا.
- 13. المحاضرة تعقد في قاعة، والقاعة تقع داخل مبنى.
- 14. الطالب يمكن أن يحضر عدة محاضرات، ويمكن للمحاضرة أن يحضرها عدة طلاب.
- 15. يتم تسجيل حالة حضور أو غياب الطالب لكل محاضرة.
- 16. المقرر يمكن أن يحتوي على عدة اختبارات، وكل اختبار مرتبط بمقرر واحد.
- 17. الاختبار يُعقد في قاعة محددة وفي فصل دراسي محدد.
- 18. لكل قسم رئيس قسم وفق قواعد الجامعة.
- 19. الطالب يمكن أن يحصل على منح دراسية وفق شروط الجامعة.
- 20. يتم تسجيل المعاملات المالية الخاصة بالطلاب.
- 21. يمكن للطلاب استخدام خدمات المكتبة.
- 22. يمكن للطلاب تقديم طلبات والاستفادة من خدمات شؤون الطلاب.
- 23. يمكن للطلاب المشاركة في مشاريع التخرج.
- 24. يمكن لعضو هيئة التدريس الإشراف على مشاريع التخرج.
- لكل Entity مفتاح أساسي Primary Key يميز كل سجل بشكل فريد.
- لا يجوز تكرار قيمة المفتاح الأساسي.
- لا يمكن أن يحتوي المفتاح الأساسي على قيمة NULL.
- المفاتيح الأجنبية Foreign Keys يجب أن تشير إلى سجلات موجودة في الجداول المرتبطة.
- لا يمكن تسجيل طالب في مقرر أو فصل دراسي غير موجود.
- لا يمكن تسجيل حضور لطلاب أو محاضرة غير موجودة.
- لا يمكن ربط اختبار بمقرر غير موجود.
- يجب أن تكون بيانات الحضور مرتبطة بالطلاب والمحاضرة.
- يجب أن تكون القاعة مرتبطة بمبنى موجود.
- يجب أن تكون المحاضرة مرتبطة بمقرر وفصل دراسي وقاعة وفق قواعد النظام.

د. مخطط المفاهيم (Conceptual ER Diagram)

يمثل مخطط المفاهيم التصور العام لنظام إدارة الجامعة قبل تحويله إلى جداول. ركزت في هذا المستوى على تحديد الكيانات الأساسية، مثل الجامعة والكليات والأقسام والتخصصات والطلاب والمقررات، وتوضيح العلاقات بينها بصورة مبسطة دون الدخول في تفاصيل أنواع البيانات أو أوامر SQL.

هـ. مخطط Relational Schema Diagram

يوضح مخطط Relational Schema Diagram تحويل الكيانات والعلاقات إلى جداول علائقية قابلة للتنفيذ. أظهرت في المخطط أسماء الجداول، الحقول، المفاتيح الأساسية PK، المفاتيح الأجنبية FK، وأنواع البيانات والقيود مثل NOT NULL كما أضفت جداول وسيطة للعلاقات متعددة-إلى-متعدد، مثل التدريس وتسجيل الطلاب في المشاريع.

شرح مبسط للصورة: تم ربط كل جدول بالجدول المرجعي الخاص به باستخدام المفاتيح الأجنبية، مما يحافظ على تكامل البيانات ويمنع إنشاء سجل مرتبط بكيان غير موجود. كما يظهر جدول الحضور بمفتاح مركب من Student_ID و Lecture_ID لتسجيل حضور الطالب في كل محاضرة.

و. الأكواد التي تم بناؤها

يتضمن هذا القسم أوامر SQL اللازمة لإنشاء قاعدة البيانات والجداول والمفاتيح والقيود والعلاقات. تم إرفاق أوامر تعريف وإنشاء البنية فقط، ولم يتم إرفاق أي أوامر INSERT أو أي بيانات تجريبية، وفقا للطلب.

-- University Management System

-- DDL only: table definitions, keys, relationships, and integrity constraints.

-- INSERT statements are intentionally excluded.

```
CREATE DATABASE UniversityManagement;
```

```
USE UniversityManagement;
```

```
CREATE TABLE University (
```

```
    University_ID INT PRIMARY KEY,
```

```
    University_Name VARCHAR(100) NOT NULL,
```

```
    University_Phone VARCHAR(20),
```

```
    University_Email VARCHAR(100)
```

```
);
```

```
CREATE TABLE Colleges (
```

```
    College_ID INT PRIMARY KEY,
```

```
    College_Name VARCHAR(100) NOT NULL,
```

```
    University_ID INT NOT NULL,
```

```
    CONSTRAINT FK_Colleges_University FOREIGN KEY (University_ID)
```

```
        REFERENCES University(University_ID)
```

```
);
```

```
CREATE TABLE Departments (
```

```
    Department_ID INT PRIMARY KEY,
```

```
    Department_Name VARCHAR(100) NOT NULL,
```

```
    College_ID INT NOT NULL,
```

```
    CONSTRAINT FK_Departments_Colleges FOREIGN KEY (College_ID)
```

```
        REFERENCES Colleges(College_ID)
```

```
);
```

```
CREATE TABLE Major (
```

```
Major_ID INT PRIMARY KEY,  
Major_Name VARCHAR(100) NOT NULL,  
Department_ID INT NOT NULL,  
CONSTRAINT FK_Major_Departments FOREIGN KEY (Department_ID)  
REFERENCES Departments(Department_ID)  
);
```

```
CREATE TABLE Students (  
Student_ID INT PRIMARY KEY,  
First_Name VARCHAR(50) NOT NULL,  
Last_Name VARCHAR(50) NOT NULL,  
Date_of_Birth DATE,  
Phone VARCHAR(20),  
Email VARCHAR(100),  
Major_ID INT NOT NULL,  
CONSTRAINT FK_Students_Major FOREIGN KEY (Major_ID)  
REFERENCES Major(Major_ID)  
);
```

```
CREATE TABLE Faculty_Members (  
Faculty_ID INT PRIMARY KEY,  
First_Name VARCHAR(50) NOT NULL,  
Last_Name VARCHAR(50) NOT NULL,  
Academic_Rank VARCHAR(50),  
Email VARCHAR(100),  
Phone VARCHAR(20),  
Department_ID INT NOT NULL,  
CONSTRAINT FK_Faculty_Departments FOREIGN KEY (Department_ID)  
REFERENCES Departments(Department_ID)  
);
```

```
CREATE TABLE Courses (  
Course_ID INT PRIMARY KEY,  
Course_Name VARCHAR(100) NOT NULL,  
Credit_Hours INT NOT NULL CHECK (Credit_Hours > 0),
```

```
Department_ID INT NOT NULL,  
  
CONSTRAINT FK_Courses_Departments FOREIGN KEY (Department_ID)  
  
REFERENCES Departments(Department_ID)  
  
);
```

```
CREATE TABLE Buildings (  
  
Building_ID INT PRIMARY KEY,  
  
Building_Name VARCHAR(100) NOT NULL,  
  
Location VARCHAR(200)  
  
);
```

```
CREATE TABLE Rooms (  
  
Room_ID INT PRIMARY KEY,  
  
Room_Number VARCHAR(20) NOT NULL,  
  
Capacity INT NOT NULL CHECK (Capacity > 0),  
  
Building_ID INT NOT NULL,  
  
CONSTRAINT FK_Rooms_Buildings FOREIGN KEY (Building_ID)  
  
REFERENCES Buildings(Building_ID)  
  
);
```

```
CREATE TABLE Academic_Semesters (  
  
Semester_ID INT PRIMARY KEY,  
  
Semester_Name VARCHAR(50) NOT NULL,  
  
Start_Date DATE NOT NULL,  
  
End_Date DATE NOT NULL,  
  
CONSTRAINT CK_Semesters_Dates CHECK (End_Date >= Start_Date)  
  
);
```

```
CREATE TABLE Lectures (  
  
Lecture_ID INT PRIMARY KEY,  
  
Lecture_Date DATE NOT NULL,  
  
Start_Time TIME NOT NULL,  
  
End_Time TIME NOT NULL,  
  
Course_ID INT NOT NULL,  
  
Room_ID INT NOT NULL,
```

```

Semester_ID INT NOT NULL,

CONSTRAINT CK_Lectures_Time CHECK (End_Time > Start_Time),

CONSTRAINT FK_Lectures_Courses FOREIGN KEY (Course_ID) REFERENCES Courses(Course_ID),

CONSTRAINT FK_Lectures_Rooms FOREIGN KEY (Room_ID) REFERENCES Rooms(Room_ID),

CONSTRAINT FK_Lectures_Semesters FOREIGN KEY (Semester_ID) REFERENCES Academic_Semesters(Semester_ID)

);

```

```

CREATE TABLE Exams (

Exam_ID INT PRIMARY KEY,

Exam_Date DATE NOT NULL,

Exam_Time TIME NOT NULL,

Course_ID INT NOT NULL,

Room_ID INT NOT NULL,

Semester_ID INT NOT NULL,

CONSTRAINT FK_Exams_Courses FOREIGN KEY (Course_ID) REFERENCES Courses(Course_ID),

CONSTRAINT FK_Exams_Rooms FOREIGN KEY (Room_ID) REFERENCES Rooms(Room_ID),

CONSTRAINT FK_Exams_Semesters FOREIGN KEY (Semester_ID) REFERENCES Academic_Semesters(Semester_ID)

);

```

```

CREATE TABLE Attendance_Absence (

Student_ID INT NOT NULL,

Lecture_ID INT NOT NULL,

Attendance_Date DATE NOT NULL,

Attendance_Status VARCHAR(20) NOT NULL,

Check_In_Time TIME,

Notes VARCHAR(255),

PRIMARY KEY (Student_ID, Lecture_ID),

CONSTRAINT FK_Attendance_Students FOREIGN KEY (Student_ID) REFERENCES Students(Student_ID),

CONSTRAINT FK_Attendance_Lectures FOREIGN KEY (Lecture_ID) REFERENCES Lectures(Lecture_ID)

);

```

```

CREATE TABLE Department_Heads (

Head_ID INT PRIMARY KEY,

Head_Name VARCHAR(100) NOT NULL,

Department_ID INT NOT NULL UNIQUE,

```

```

CONSTRAINT FK_Heads_Departments FOREIGN KEY (Department_ID)

REFERENCES Departments(Department_ID)

);

```

```

CREATE TABLE Scholarships (

    Scholarship_ID INT PRIMARY KEY,

    Scholarship_Name VARCHAR(100) NOT NULL,

    Amount DECIMAL(10,2) NOT NULL CHECK (Amount >= 0),

    Start_Date DATE,

    End_Date DATE,

    Student_ID INT NOT NULL,

    CONSTRAINT FK_Scholarships_Students FOREIGN KEY (Student_ID)

    REFERENCES Students(Student_ID)

);

```

```

CREATE TABLE Finance (

    Finance_ID INT PRIMARY KEY,

    Student_ID INT NOT NULL,

    Amount DECIMAL(10,2) NOT NULL,

    Transaction_Date DATE NOT NULL,

    Transaction_Type VARCHAR(50) NOT NULL,

    CONSTRAINT FK_Finance_Students FOREIGN KEY (Student_ID)

    REFERENCES Students(Student_ID)

);

```

```

CREATE TABLE Library (

    Library_ID INT PRIMARY KEY,

    Library_Name VARCHAR(100) NOT NULL,

    Location VARCHAR(200)

);

```

```

CREATE TABLE Student_Affairs (

    Affairs_ID INT PRIMARY KEY,

    Student_ID INT NOT NULL,

    Request_Type VARCHAR(100) NOT NULL,

```

```

Request_Date DATE NOT NULL,

Status VARCHAR(30) NOT NULL,

CONSTRAINT FK_Affairs_Students FOREIGN KEY (Student_ID)

REFERENCES Students(Student_ID)

);

```

```

CREATE TABLE Graduation_Projects (

Project_ID INT PRIMARY KEY,

Project_Name VARCHAR(150) NOT NULL,

Project_Description VARCHAR(500),

Project_Date DATE

);

```

```

CREATE TABLE Academic_Registration (

Registration_ID INT PRIMARY KEY,

Student_ID INT NOT NULL,

Course_ID INT NOT NULL,

Semester_ID INT NOT NULL,

Registration_Date DATE NOT NULL,

CONSTRAINT UQ_Registration UNIQUE (Student_ID, Course_ID, Semester_ID),

CONSTRAINT FK_Registration_Students FOREIGN KEY (Student_ID) REFERENCES Students(Student_ID),

CONSTRAINT FK_Registration_Courses FOREIGN KEY (Course_ID) REFERENCES Courses(Course_ID),

CONSTRAINT FK_Registration_Semesters FOREIGN KEY (Semester_ID) REFERENCES Academic_Semesters(Semester_ID)

);

```

```

CREATE TABLE Faculty_Course_Semester (

Faculty_ID INT NOT NULL,

Course_ID INT NOT NULL,

Semester_ID INT NOT NULL,

PRIMARY KEY (Faculty_ID, Course_ID, Semester_ID),

CONSTRAINT FK_Teaching_Faculty FOREIGN KEY (Faculty_ID) REFERENCES Faculty_Members(Faculty_ID),

CONSTRAINT FK_Teaching_Courses FOREIGN KEY (Course_ID) REFERENCES Courses(Course_ID),

CONSTRAINT FK_Teaching_Semesters FOREIGN KEY (Semester_ID) REFERENCES Academic_Semesters(Semester_ID)

);

```

```
CREATE TABLE Student_Project (  
  
    Student_ID INT NOT NULL,  
  
    Project_ID INT NOT NULL,  
  
    PRIMARY KEY (Student_ID, Project_ID),  
  
    CONSTRAINT FK_StudentProject_Students FOREIGN KEY (Student_ID) REFERENCES Students(Student_ID),  
  
    CONSTRAINT FK_StudentProject_Projects FOREIGN KEY (Project_ID) REFERENCES Graduation_Projects(Project_ID)  
  
);
```

```
CREATE TABLE Faculty_Project_Supervision (  
  
    Faculty_ID INT NOT NULL,  
  
    Project_ID INT NOT NULL,  
  
    PRIMARY KEY (Faculty_ID, Project_ID),  
  
    CONSTRAINT FK_Supervision_Faculty FOREIGN KEY (Faculty_ID) REFERENCES Faculty_Members(Faculty_ID),  
  
    CONSTRAINT FK_Supervision_Projects FOREIGN KEY (Project_ID) REFERENCES Graduation_Projects(Project_ID)  
  
);
```